阿里云 × CLOUD NATIVE COMPUTING FOUNDATION

云原生技术公开课

第 18 讲

# Kubernetes调度和资源管理

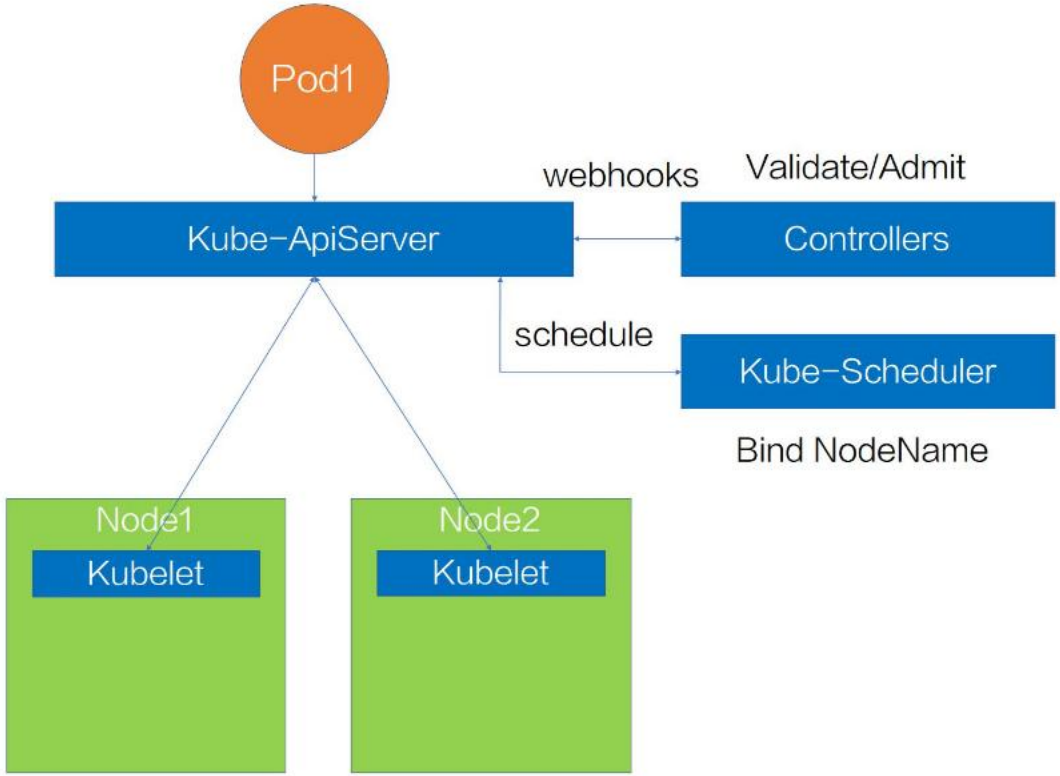
子誉 蚂蚁金服高级技术专家

# 1 Kubernetes 调度过程

# 目录

1. Kubernetes 调度过程
2. Kubernetes 基础调度能力
   - 资源调度
   - 关系调度
3. Kubernetes 高级调度能力
   - PodPriority/Preemption – 优先级和抢占
4. 调度器架构简介和具体算法（下一节介绍）

# Kubernetes 调度过程




调度过程 = 把Pod放到合适Node上去

什么是合适?

- 满足Pod资源要求
- 满足Pod的特殊关系要求
- 满足Node限制条件要求
- 做到集群资源合理利用

# 2 Kubernetes 基础调度能力

# Kubernetes 基础调度能力

- 资源调度 – 满足Pod资源要求
  - Resources：CPU/Memory/Storage/GPU/FGPA
  - QoS： Guaranteed/Burstable/BestEffort
  - Resource Quota
- 关系调度 – 满足Pod/Node的特殊关系/条件要求
  - PodAffinity/PodAntiAffinity： Pod和Pod间关系
  - NodeSelector/NodeAffinity： 由Pod决定适合自己的Node
  - Taint/Tolerations： 限制调度到某些Node

# 如何满足Pod资源要求？ –资源调度用法

- Pod的资源类型
  - cpu
  - memory
  - ephemeral-storage
  - extended-resource: nvidia.com/gpu
- 怎么用?
  - Container resources
  - request/limit 最小/最大
  - Cpu: 1=1000m
  - mem/storage: 1Gi=1024Mi
  - extended-resource: 整数

```
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
    resources:
      requests:
        cpu: 2
        memory: 1Gi
      limits:
        cpu: 2
        memory: 1Gi
```

# 如何满足Pod资源要求？ – 资源QoS类型

- 为什么会有request和limit?
- 什么是Pod QoS?
  - Quality of Service
- 三类QoS
  - Guaranteed – 高，保障
  - Burstable – 中，弹性
  - BestEffort – 低，尽力而为
- 隐性的QoS Class

```
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
    resources:
      requests:
        cpu: 2
        memory: 1Gi
      limits:
        cpu: 2
        memory: 1Gi
status:
  qosClass: Guaranteed
```

# 如何满足Pod资源要求？ – 资源QoS用法

怎么用?

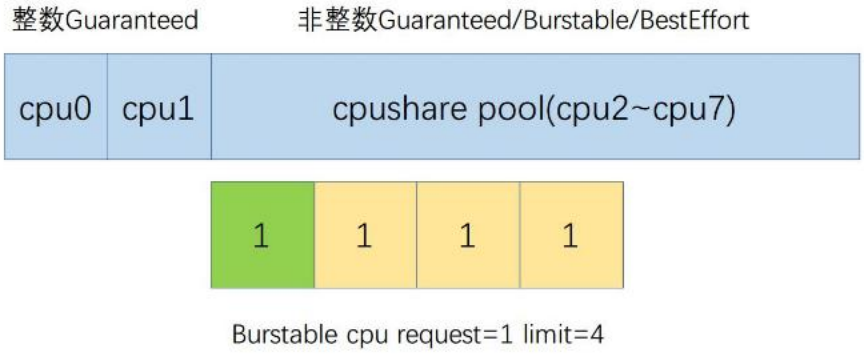
- Guaranteed: CPU/Memory 必须request==limit，其余资源可不等
- Burstable: CPU/Memory request和limit不相等
- BestEffort: 所有资源request/limit必须都不填

1 Guaranteed Pod：
1 Burstable Pod：
1 BestEffort Pod：

```
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-burstable-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
      requests:
        cpu: 1
      limits:
        cpu: 2
        memory: 1Gi
```

# 如何满足Pod资源要求？ – 资源和QoS

- 不同QoS有什么区别呢?
- 调度表现不同
  - 调度器会使用request进行调度
- 底层表现不同
  - CPU 按照request划分权重
    - --cpu-manager-policy=static guaranteed整数会绑核
  - Memory 按QoS划分OOMScore
    - Guaranteed -998
    - Burstable 2~999
    - BestEffort 1000
  - Eviction
    - 优先BestEffort
    - Kubelet – CPUManager



# 如何满足Pod资源要求？ – 资源Quota

- ResourceQuota
- 限制每个Namespace资源用量
- 用法
  - per namespace
  - Scope:
    - Terminating/NotTerminating
    - BestEffort/NotBestEffort
    - PriorityClass
- 当Quota用超过后会禁止创建:
  - forbidden: exceeded quota

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: demo-quota
  namespace: demo-ns
spec:
  hard:
    cpu: "1000"
    memory: 200Gi
    pods: "10"
  scopeSelector:    # 可以不填
    matchExpressions:
    - operator: Exists
      scopeName: NotBestEffort
```

限制demo-ns Namespace下非BestEffort QoS的Quota

- cpu只能使用1000个
- memory只能使用200Gi
- pods只能创建10个

# 小结：如何满足Pod资源要求?

- Pod要配置合理的资源要求
  - CPU/Memory/EphemeralStorage/GPU
- 通过request和limit来为不同业务特点的Pod选择不同的QoS
  - Guaranteed – 敏感型、需要保障业务
  - Burstable – 次敏感型、需要弹性业务
  - BestEffort – 可容忍型业务
- 为每个NS配置ResourceQuota来防止过量使用，保障其他人的资源可用

# 如何满足Pod与Pod关系要求？ – Pod亲和调度

- Pod亲和调度 *PodAffinity*
  - 必须和某些Pod调度到一起

    requiredDuringSchedulingIgnoredDuringExecution
  - 优先和某些Pod调度到一起

    preferredDuringSchedulingIgnoredDuringExecution
- Pod反亲和调度 *PodAntiAffinity*
  - 禁止和某些Pod调度到一起

    requiredDuringSchedulingIgnoredDuringExecution
  - 优先不和某些Pod调度到一起

    preferredDuringSchedulingIgnoredDuringExecution
- Operator
  - In/NotIn/Exists/DoesNotExist

优先调度到带了k2=v2标签的Pod所在节点：

```yaml
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: k2
              operator: In
              values:
              - v2
          topologyKey: "kubernetes.io/hostname"
```

# 如何满足Pod与Node关系要求？ – Node亲和调度

- *NodeSelector*
  - 必须调度到带了某些标签的Node
  - Map[string]string
- *NodeAffinity*
  - 必须调度到某些Node上
    requiredDuringSchedulingIgnoredDuringExecution
  - 优先调度到某些Node上
    preferredDuringSchedulingIgnoredDuringExecution
  - Operator
    - In/NotIn/Exists/DoesNotExist/Gt/Lt

优先调度到带了k2=v2标签的Node上：

```yaml
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 10
        preference:
          matchExpressions:
          - key: k2
            operator: In
            values:
            - v2
```


阿里云 CLOUD NATIVE COMPUTING FOUNDATION

# 如何限制调度到某些Node？ – Node 标记/容忍

- Node *Taints*
  - 一个Node可以有多个Taints
  - Effect（不能为空）
    - NoSchedule – 只是禁止新Pods调度上来
    - PreferNoSchedule – 尽量不调度到这台
    - NoExecute – 会evict没有对应toleration的Pods，并且也不会调度新的上来
- Pod *Tolerations*
  - 一个Pod可以有多个Tolerations
  - Effect可以为空，匹配所有
    - 取值和Taints的Effect一致
  - Operator
    - Exists/Equal

Node上带了k1=v1且效果是NoSchedule的Taints：

```yaml
apiVersion: v1
kind: Node
metadata:
  name: demo-node
spec:
  taints:
    - key: "k1"
      value: "v1"
      effect: "NoSchedule"
```

Pod上必须带有k1=v1且效果是NoSchedule的Tolerations：

```yaml
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
  tolerations:
  - key: "k1"
    operator: "Equal"
    value: "v1"                    # 当op=Exists可为空
    effect: "NoSchedule"           # 可以为空，匹配所有
```



阿里云 CLOUD NATIVE COMPUTING FOUNDATION

# 小结：如何满足Pod/Node的特殊关系/条件要求?

- 给Pod配置和其他Pod的亲和/互斥关系
  - *PodAffinity*
  - *PodAntiAffinity*
- 给Pod配置和Node的亲和关系
  - *NodeSelector*
  - *NodeAffinity*
- 给Node配置一些限制条件标记，给Pods配置容忍标记
  - *Node - Taints*
  - *Pod - Tolerations*



100%
阿里云
CLOUD NATIVE COMPUTING FOUNDATION

# 3 Kubernetes 高级调度能力



阿里云 CLOUD NATIVE COMPUTING FOUNDATION

# Kubernetes 高级调度能力

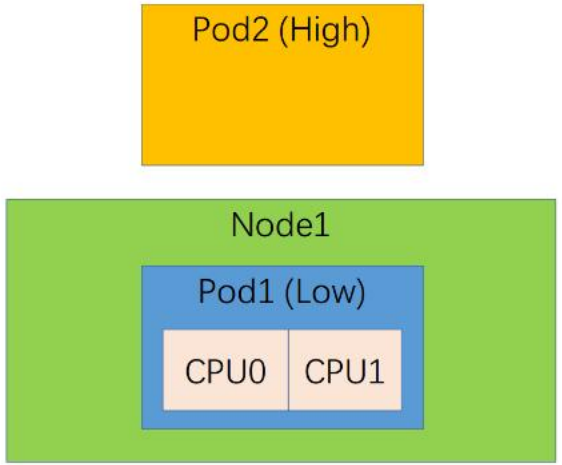
- 优先级调度和抢占
  - Priority
  - Preemption



阿里云 CLOUD NATIVE COMPUTING FOUNDATION

# 如何做到集群资源合理利用？ – 优先级调度

- 集群资源不够用时候怎么办?
  - 先到先得策略（FIFO） – 简单、相对公平、上手快
  - 优先级策略（Priority） – 符合日常公司业务特点
- PodPriority和Preemption
  - v1.14 – stable
  - PodPriority & Preemption default is ON



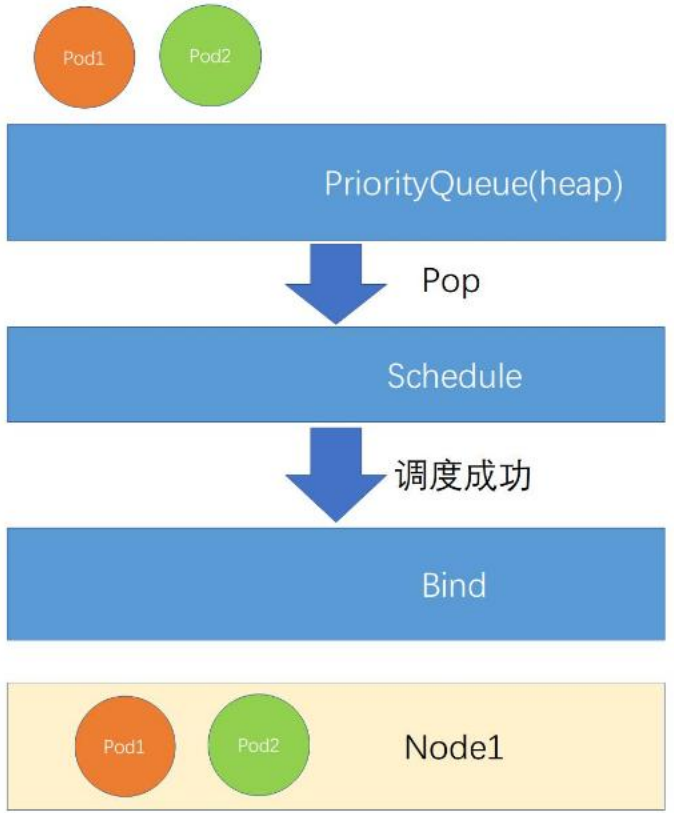


阿里云
CLOUD NATIVE COMPUTING FOUNDATION

# 如何做到集群资源合理利用？ – 优先级调度过程

优先级调度过程（发生在Pending Pod出队列开始调度）：

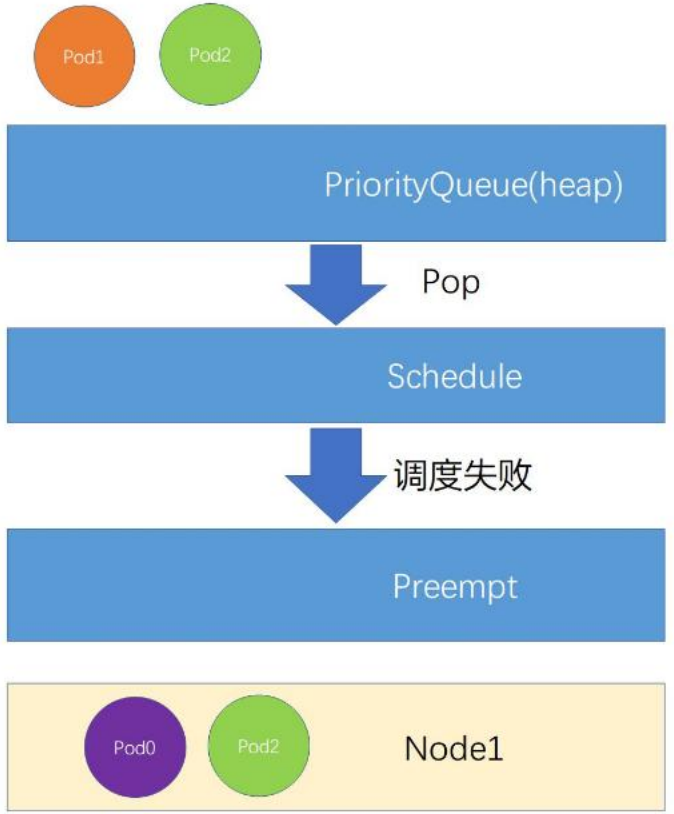
1. Pod2和Pod1先后进入调度队列，但均未被调度
2. 当进行一轮调度时，PriorityQueue会优先Pop优先级更大的Pod1出队列镜像调度
3. 调度成功后，对此Pod1进行Bind，并开始下一轮调度Pod2



# 如何做到集群资源合理利用？ – 优先级抢占过程

优先级抢占过程（发生在调度失败）：

1. Pod2先进行调度，调度成功后分配至Node1上运行
2. 之后Pod1再进行调度，由于Node1资源不足出现了调度失败，此时进入抢占流程
3. 在经过抢占算法计算后，选中了Pod2作为Pod1的让渡者
4. 驱逐了Node1上运行的Pod2，并将Pod1调度至Node1



# 谢谢观看

THANK YOU